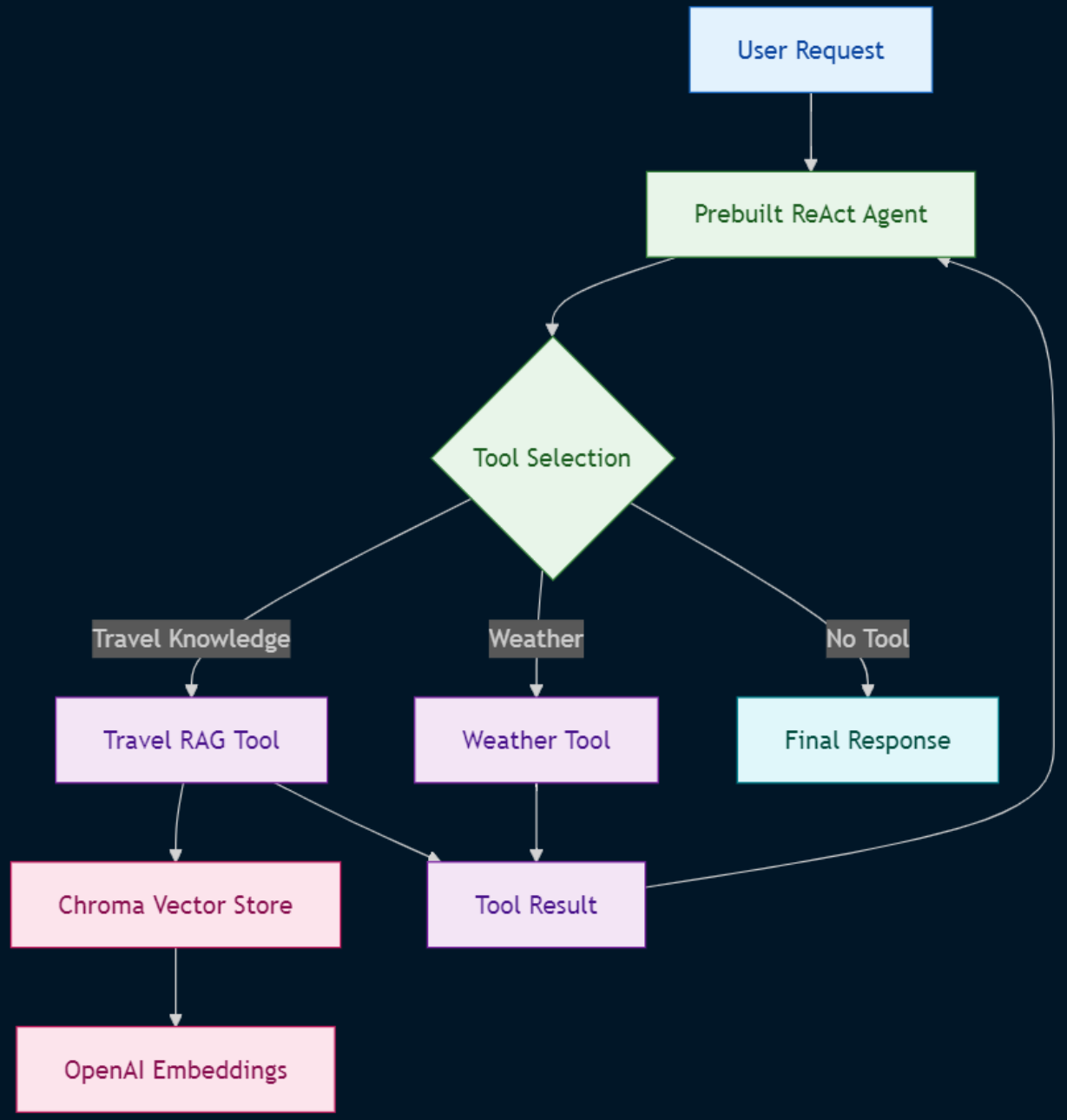
Agentic AI Engineering

# *LangGraph Prebuilt ReAct Agent with RAG and Multi-Tool Orchestration*

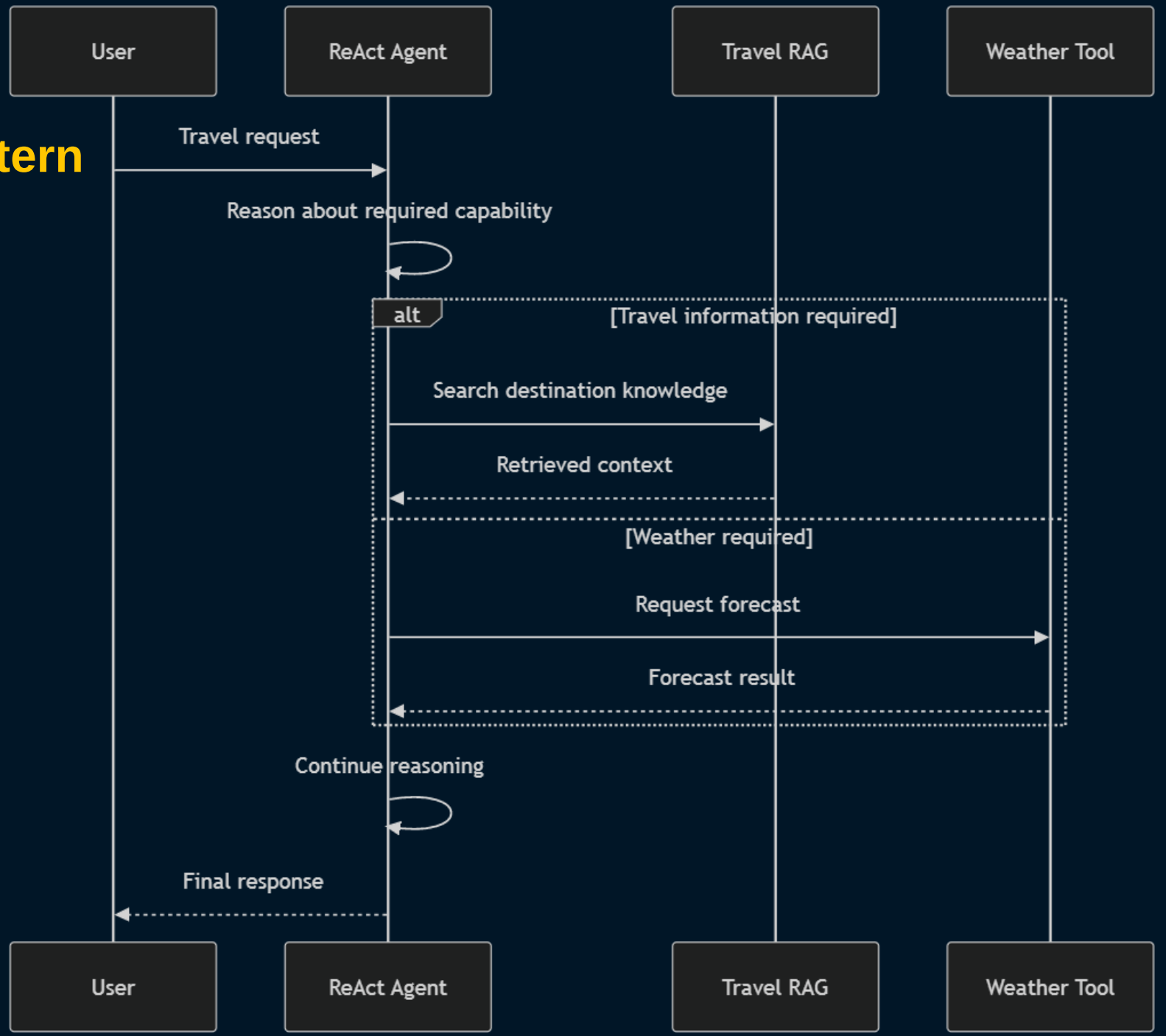
*Max Montaseri*

- Architected a **prebuilt ReAct Agentic AI workflow** using LangChain create_agent, reducing custom graph/tool-orchestration boilerplate while retaining multi-tool reasoning capabilities.
- Built a RAG subsystem using **AsyncHtmlLoader**, **recursive document chunking**, **OpenAI embeddings** and **Chroma vector search** for destination-specific travel knowledge.
- Integrated multiple agent tools for **semantic travel retrieval and weather forecasting**, enabling dynamic LLM-driven capability selection.
- Designed typed agent state using **TypedDict** and **RemainingSteps** to support controlled tool-calling execution and bounded agent loops.
- Migrated from manually wired LangGraph nodes and custom tool execution to a higher-level agent abstraction, demonstrating architectural trade-offs between **control**, **maintainability** and **development velocity**.
- Integrated **LangSmith observability** for tracing and inspection of agent execution, tool calls, LLM interactions, and intermediate workflow behavior.

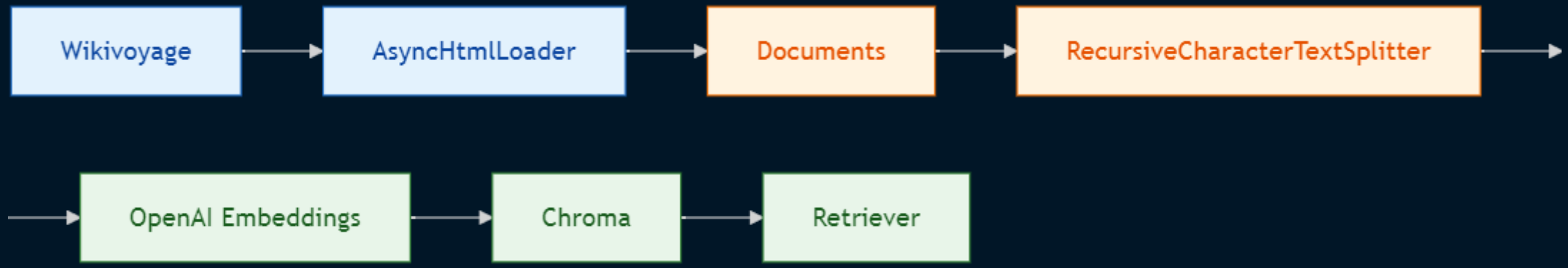
# Architecture



# ReAct Agent Pattern



# Travel Knowledge Base and RAG
# The ingestion process




The application builds a travel knowledge base from Wikivoyage pages for:

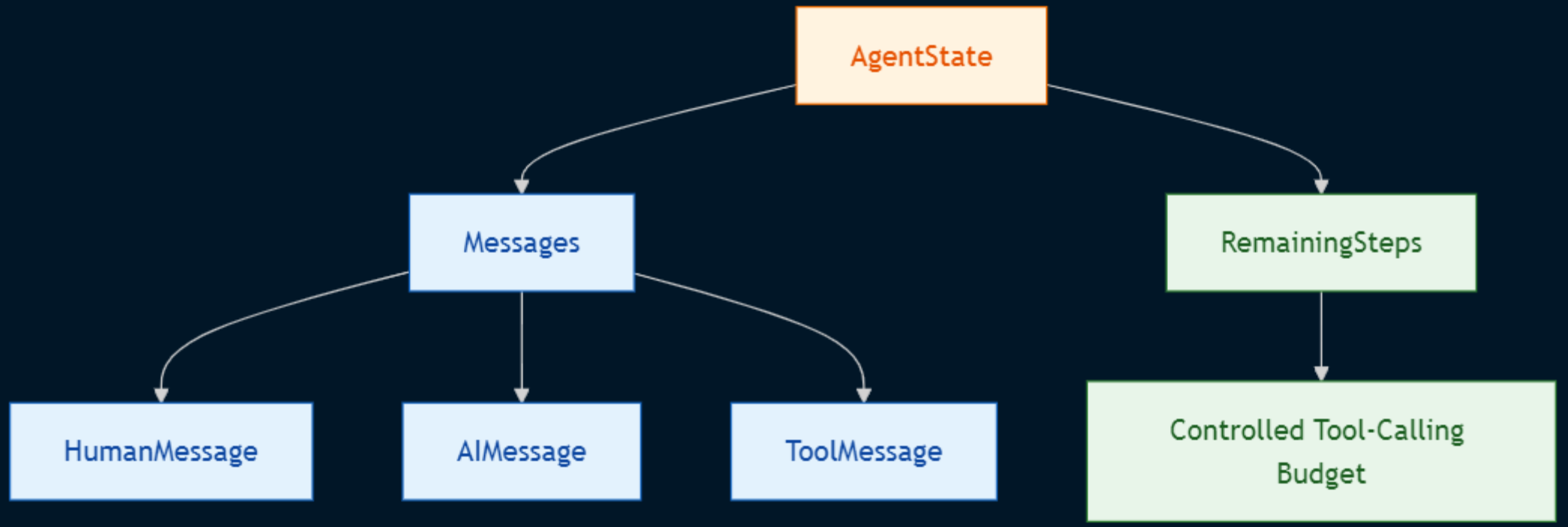
- Cornwall
- North Cornwall
- South Cornwall
- West Cornwall

Current Configuration

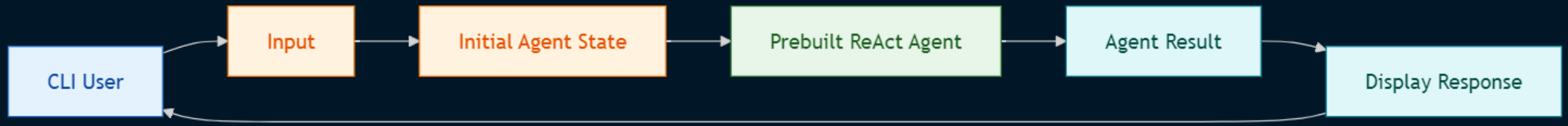
- Asynchronous HTML loading
- 1024-character chunks
- 128-character overlap
- OpenAI embeddings
- Chroma vector storage
- Retriever abstraction

The vector store is initialized through a cached module-level client.

# State Model


AgentState
Messages
RemainingSteps
HumanMessage
AIMessage
ToolMessage
Controlled Tool-Calling Budget

# CLI Application Flow



# Recommended Production Evolution